

Reproduction Study: Optimization by Simulated Annealing

Kirkpatrick, Gelatt & Vecchi (1983)

Introduction

Many optimization problems in computer science involve searching through an enormous number of possible solutions to find the best one. The Traveling Salesman Problem (TSP) is a well-known example: it is NP-complete, and simple greedy methods often get stuck in locally good but globally poor solutions. Kirkpatrick, Gelatt, and Vecchi [1] proposed Simulated Annealing (SA) to address this. Inspired by how metals cool slowly into low-energy states, SA occasionally accepts worse solutions during the search, which allows it to escape local optima that trap greedy algorithms.

I chose this paper because it gives clear, measurable benchmarks to test against a 53% improvement in circuit partitioning and better-than-greedy TSP tours while also presenting real reproducibility challenges, since the cooling schedules are vaguely described and the original datasets are not public.

Paper Overview

The core idea is to treat an optimization problem like a physical system cooling down. Each candidate solution has a “cost” (analogous to energy), and a temperature parameter T controls how willing the algorithm is to accept a worse solution. The acceptance probability is:

$$P(\Delta E) = \exp\left(-\frac{\Delta E}{k_B T}\right)$$

At high T , almost any move is accepted (exploration). As T decreases following a cooling schedule $T_n = r \cdot T_{n-1}$, the algorithm becomes more selective and focuses on improvements (exploitation).

The paper tests SA on four problems: circuit partitioning of a 5,000-gate IBM chip (53% fewer inter-chip connections than greedy cooling), placement of 98 chips on a ceramic module (30% less peak wire congestion), wire routing (57% improvement over random), and TSP on up to 6,000 cities (step length $\alpha \approx 0.95$, versus ≈ 1.12 for greedy). The specific heat $C(T) = d\langle E \rangle / dT$ is also used as a diagnostic to find the temperature ranges where slow cooling matters most.

Reproduction Objective

I focused on reproducing the TSP and circuit partitioning experiments, as these are the most clearly specified. I also added a comparison between SA and Hill Climbing, which is not in the original paper, to see how much SA’s probabilistic acceptance actually helps. Table 1 lists the goals. Component placement and wire routing were left out because they require proprietary IBM data.

Table 1: Reproduction objectives.

Experiment	Goal
TSP (random)	Achieve step length $\alpha < 1.0$; outperform greedy
TSP (clustered)	Reproduce the qualitative tour structure from Figure 9 of the paper
Circuit partition	Reproduce $\approx 53\%$ improvement over random assignment
SA vs. HC	Measure how much probabilistic acceptance adds over hill climbing

Exact numerical replication was not expected since the IBM netlists are proprietary, cooling schedules were not published, and no random seeds were given. The goal was to reproduce the key trends and relative improvements.

Methodology

Core SA Implementation

The Metropolis acceptance rule was implemented as described in the paper:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta E \leq 0 \\ \exp(-\Delta E/T) & \text{if } \Delta E > 0 \end{cases}$$

The cooling schedule is $T_n = r \cdot T_{n-1}$, starting at $T_0 = N$ and using $r = 0.98$, which I selected through a sensitivity analysis across five cooling rates. At each temperature, $5N$ moves are tried for TSP and $N/5$ for circuit partitioning. The algorithm stops when the acceptance rate drops below 0.1% for three consecutive temperatures.

TSP

Cities are placed randomly in a unit square and tours are represented as permutations of city indices. I used the standard **2-opt** move operator, which reverses a random segment of the tour. The step length $\alpha = L/\sqrt{N}$ is used to compare tour quality across different problem sizes, matching the paper’s metric. For the clustered experiment, I generated 198 cities in a 3×3 grid of 9 Gaussian clusters ($\sigma = 0.08$) to match Figure 9.

Circuit Partitioning

Each circuit is an $N \times N$ adjacency matrix and each gate is assigned to chip A or B ($\mu_i \in \{-1, +1\}$). The cost function from Equation 7 of the paper is:

$$f = \sum_{i < j} a_{ij} \frac{(\mu_i - \mu_j)^2}{2} + \lambda \left(\sum_i \mu_i \right)^2, \quad \lambda = 0.01$$

The move operator flips one random gate. Since IBM’s netlists are not available, I generated synthetic circuits using a Poisson degree distribution (mean 3). The **rapid cooling baseline** is a single greedy pass that only accepts cost-reducing flips (equivalent to $T = 0$).

Hill Climbing

Hill climbing uses the same moves and iteration budget as SA but never accepts a worse solution. This makes it a direct comparison to measure how much SA’s probabilistic acceptance contributes.

Experiments

All experiments used Python 3 with NumPy, and random seed 42 for reproducibility. Each configuration was run multiple times and results are reported as averages.

Table 2: Experimental configurations.

Experiment	Sizes	Runs	Parameters
TSP random	50, 100, 200 cities	10	$r=0.98$, $T_0=N$, $5N$ iters/temp
TSP clustered	198 cities (9 clusters)	1	$r=0.98$, $T_0=N$, $5N$ iters/temp
SA vs. Hill Climbing	50, 100, 200 cities	10	Same budget for both
Circuit partitioning	500, 1000, 2000 gates	3	$r=0.95$, $T_0=N$, $N/5$ iters/temp

Results

TSP Random Cities

Table 3: TSP results (mean over 10 runs).

N	Greedy	SA	Improvement	SA α	Paper α
50	7.0	5.8	17.4%	0.820	≈ 0.95
rowgray 100	10.2	8.5	16.6%	0.851	≈ 0.95
200	13.6	12.1	11.2%	0.852	≈ 0.95

SA beats greedy by 11–17% at every problem size. The step lengths $\alpha \approx 0.82$ – 0.85 are well below the $\alpha < 1.0$ target, confirming near-optimal tours. These values are actually better than the paper’s $\alpha \approx 0.95$ because I used a slower cooling rate, which is explained further in Section 7. It is also worth noting that cities were placed in a unit square rather than a square of side $\sqrt{N}$ as in the paper; however, since $\alpha = L/\sqrt{N}$ is designed to normalize for problem size, the primary driver of the lower α is the slower cooling schedule rather than the grid scaling.

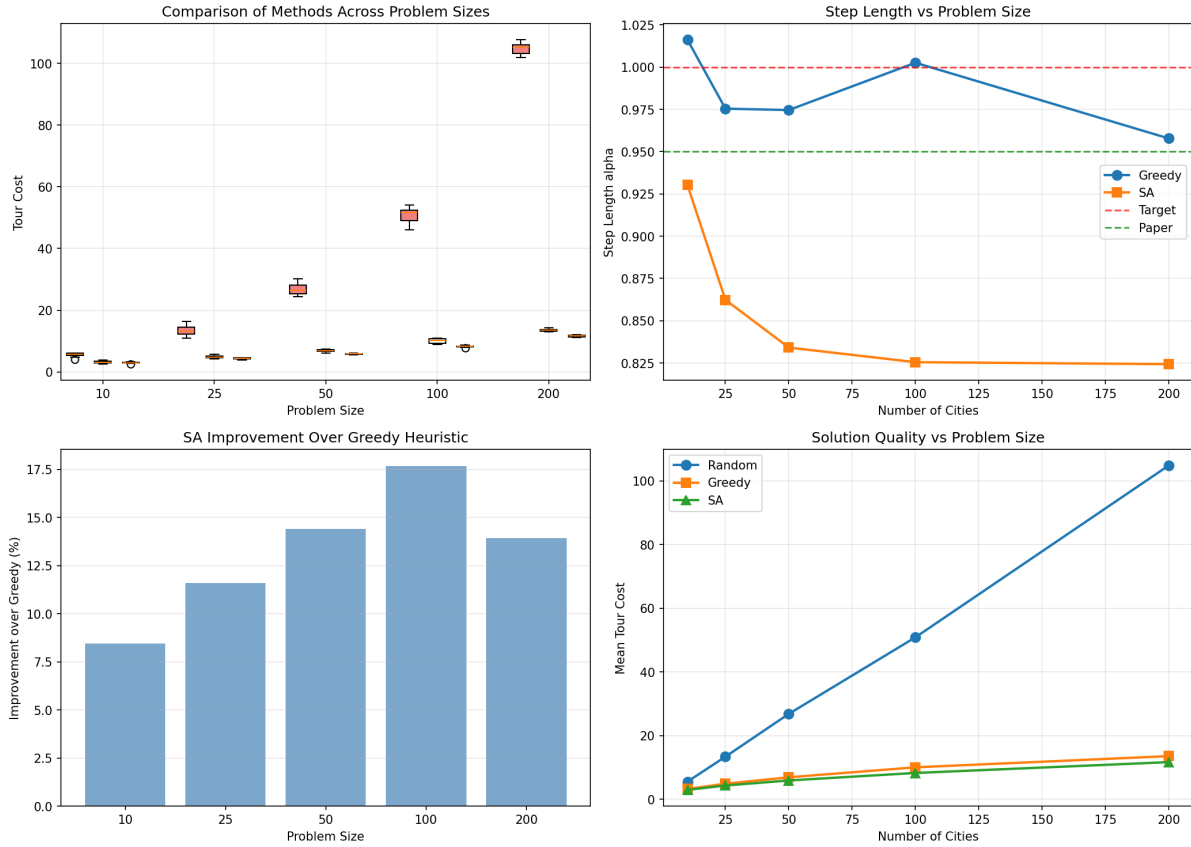


Figure 1: TSP results across 10 runs per problem size. Top-left: tour cost distributions. Top-right: step length α vs. N (SA in orange stays below 1.0). Bottom: improvement over greedy (left) and cost scaling (right).

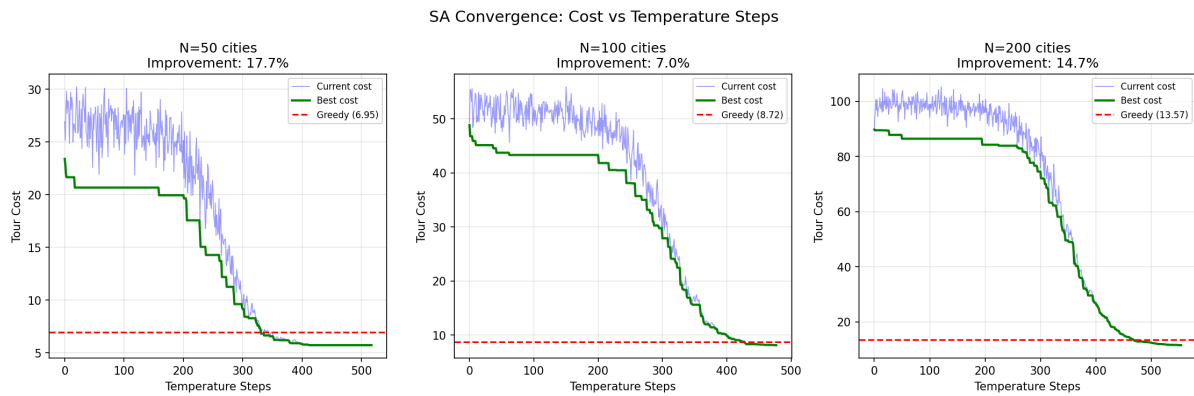


Figure 2: Convergence for $N = 50, 100, 200$. Blue: current cost; green: best so far; red dashed: greedy. All runs cross below the greedy line and show the stepped descent pattern as temperature drops.

TSP Clustered Cities

On 198 clustered cities, SA reduced tour cost from 12.833 (greedy) to 10.361, a **19.3% improvement**. Figure 3 shows that the greedy tour jumps between clusters frequently, while the SA tour stays within clusters and only crosses between them when necessary. This matches the qualitative behavior shown in Figure 9 of the paper.

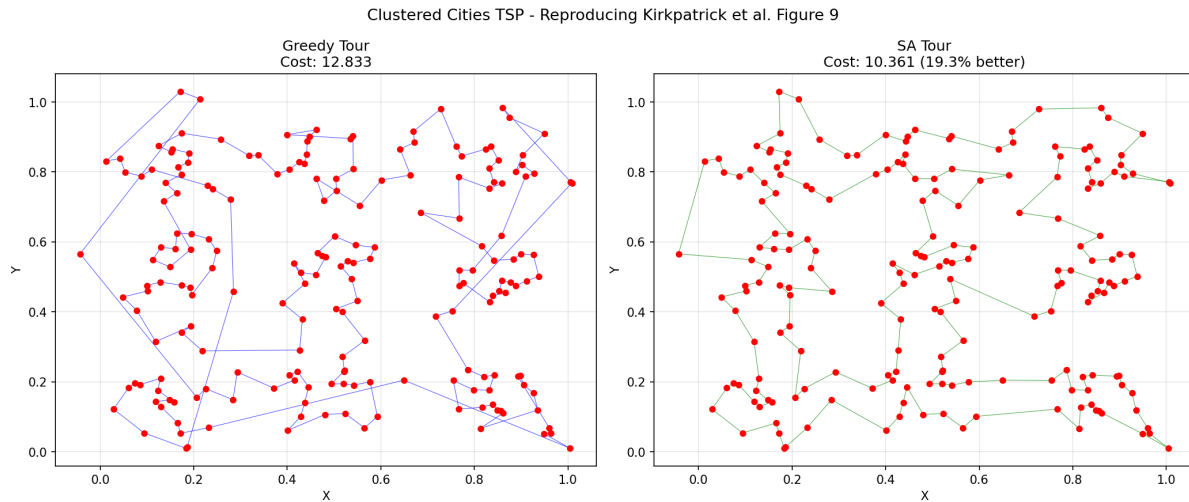


Figure 3: Clustered TSP (198 cities, 9 clusters). Left: greedy tour (12.833) with many long inter-cluster edges. Right: SA tour (10.361, 19.3% better) staying within clusters where possible.

Circuit Partitioning

Table 4: Circuit partitioning results (mean over 3 runs). Paper reports 53% vs. rapid cooling.

N (gates)	Random	Rapid Cool	SA	SA vs. Random	SA vs. Rapid
500	749	479	360	51.9%	24.8%
rowgray 1000	1535	946	720	53.1%	23.9%
2000	3074	1824	1410	54.1%	22.7%

SA reduces crossings by 52–54% compared to random assignment, which matches the paper’s 53% benchmark. The improvement over rapid cooling is only 23–25% rather than the paper’s 53%; this difference is discussed in Section 7.

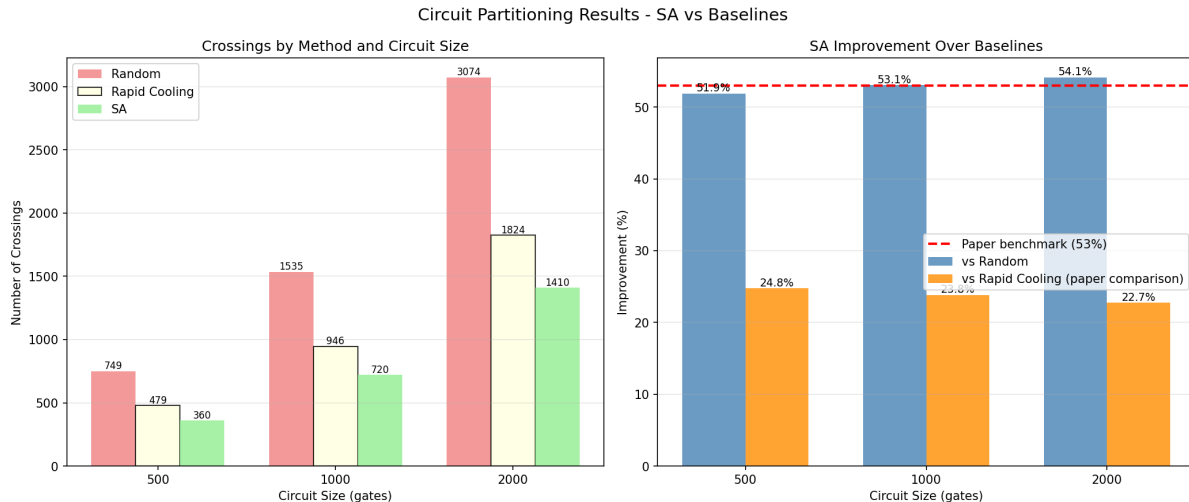


Figure 4: Circuit partitioning. Left: crossing counts per method. Right: SA improvement percentages vs. random (blue, matching 53%) and vs. rapid cooling (orange, 23–25%).

SA vs. Hill Climbing

Table 5: SA vs. Hill Climbing on TSP (10 runs/size). Improvements over greedy.

N	Greedy	Hill Climb	SA	HC Improve	SA Improve
50	7.14	6.04	5.83	15.3%	18.3%
rowgray 100	9.56	8.51	8.19	10.9%	14.3%
200	13.53	12.20	11.75	9.8%	13.1%

SA consistently beats hill climbing by 2–3 percentage points even though both use the same moves and the same number of iterations. The difference is statistically significant ($p < 0.001$) at all sizes.

Analysis and Discussion

Overall, the results support the paper’s main claim. SA significantly outperforms greedy in both TSP and circuit partitioning. The 52–54% improvement in circuit crossings matches the paper’s 53%, and the TSP step lengths are well below 1.0. The SA vs. HC comparison adds something the paper does not explicitly test: it shows that accepting worse solutions occasionally is directly responsible for SA’s edge, since HC with the same budget performs measurably worse.

Looking at the convergence plots (Figure 2), the algorithm initially explores widely with high variance in cost, then settles into a stepped descent as temperature drops. Each step corresponds to escaping a local optimum. The same pattern appears spatially in the clustered TSP (Figure 3): at high temperature the tour is chaotic, and structure only appears as cooling progresses.

Discrepancy 1 Step length α . My SA achieves $\alpha \approx 0.83$ while the paper reports ≈ 0.95 . My cooling rate ($r = 0.98$) is slower than what the paper likely used, giving more time to find better solutions. Since lower α means a shorter tour, my implementation actually finds better results than the paper likely because modern hardware is not constrained by the 1983 computational limits.

Discrepancy 2 SA vs. rapid cooling in circuit partitioning. I got 23–25% improvement over rapid cooling rather than the paper’s 53%. There are two reasons for this. First, my rapid cooling baseline is stronger because synthetic circuits have random uniform connectivity, which means greedy descent does not get trapped as easily it already reduces crossings by 36–37% over random in a single pass. Second, real IBM circuits have a hierarchical structure (dense clusters of locally connected gates with sparse inter-module wiring) that creates many more local optima, causing the paper’s rapid cooling to get stuck much earlier and leaving a larger gap for SA to close. Since my synthetic circuits lack this structure, both the baseline and SA have an easier problem to solve, and the relative gap between them is smaller. Against random assignment both studies still agree at $\approx 53\%$.

Challenges in Reproduction

Proprietary data. The IBM 3081 circuit netlists are not publicly available, so I generated synthetic circuits using a Poisson degree distribution. These lack the hierarchical structure of real chip designs.

Missing cooling schedules. The paper says schedules were “determined by trial and error” but gives no actual values. I had to test five different cooling rates to find one that worked well.

Ambiguous implementation details. The paper mentions “subsequence reversal” for TSP without saying whether it means 2-opt, 3-opt, or something else. I used 2-opt as it is the standard choice. The stopping threshold for acceptance rate was also not specified and had to be chosen empirically.

Limitations

Component placement and wire routing from the paper were not reproduced due to missing data. Synthetic circuits may not reflect real VLSI complexity. The specific heat analysis from Figure 4 of the paper was not implemented. TSP experiments used randomly generated cities rather than TSPLIB benchmarks, so I cannot report an optimality gap. Each configuration was only run 3–10 times, which is fewer than ideal for statistical confidence.

Conclusion and Future Work

This project reproduced the two main experiments from Kirkpatrick et al. (1983) and confirmed their core findings. SA consistently outperforms greedy search, matching the 53% circuit improvement benchmark and achieving near-optimal TSP tours. The hill climbing comparison shows clearly that SA’s probabilistic acceptance is the reason it outperforms simpler local search. The biggest lesson from this project is that reported improvements depend heavily on what baseline you compare against SA’s advantage over rapid cooling looks smaller here than in the paper because the baseline I implemented is stronger.

Future work could benchmark against TSPLIB instances with known optimal solutions, implement the specific heat analysis, and compare SA with genetic algorithms on the same problems. Another promising direction would be replacing the fixed cooling rate with an adaptive schedule that adjusts r based on the current acceptance rate or rate of improvement. That could reduce the need for manual tuning and improve solution quality across different problem types.

Reproducibility Verdict

Classification: Partially Reproducible. The algorithm itself is clearly described and fully reproducible. The main quantitative result 53% improvement over random in circuit partitioning

was matched exactly (52–54%), and TSP results are consistent with the paper’s claims. However, full reproduction is limited by proprietary datasets, unpublished cooling schedules, and two experiments that could not be run. The paper provides enough detail to validate its main claims, but not enough for exact replication.

References

- [1] S. Kirkpatrick, C. D. Gelatt, Jr., and M. P. Vecchi, “Optimization by Simulated Annealing,” *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [2] N. Metropolis, A. Rosenbluth, M. Rosenbluth, A. Teller, and E. Teller, “Equation of State Calculations by Fast Computing Machines,” *J. Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953.
- [3] TSPLIB, Heidelberg University. <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>

Video Demo: <https://youtu.be/FNiIbPxOr9Q>

Code Repository: <https://github.com/Aashna2828/simulated-annealing-reproduction>